

ReLU Performance Analysis on GPUs: A Comparison of PyCUDA and PyOpenCL

Nrushad Joshi
Columbia University
naj2139@columbia.edu

Abstract—This report evaluates the performance of an element-wise ReLU kernel implemented in CUDA (via PyCUDA) and OpenCL (via PyOpenCL) on an NVIDIA Tesla P4 GPU, benchmarked against a NumPy CPU baseline for vector lengths from 10 to 10^8 . For each API, a high-level array interface is compared with explicit memory management, and kernel-only time is measured separately from end-to-end time. The GPU kernel outperforms NumPy from $N \approx 10^5$ in PyCUDA and at all tested sizes in PyOpenCL. However, host-device data transfer accounts for most of the end-to-end runtime, so NumPy remains faster end-to-end across the entire tested range.

I. INTRODUCTION

GPUs offer massive parallelism for data-parallel workloads, but offloading work to them comes with overhead: memory allocation, host-device data transfers, and kernel launch all add to the total runtime. For simple, memory-bound operations, these fixed and per-byte costs can outweigh the computational speedup. ReLU, $y_i = \max(x_i, 0)$, is a natural test case: it performs one comparison per element loaded and stored, so it isolates the cost of moving data from the cost of computing on it.

This work quantifies that trade-off on a common cloud GPU. It compares two APIs (CUDA and OpenCL) and two memory-management styles within each, and it separates kernel-only time from end-to-end time.

II. METHODOLOGY

A. Implementation

Both kernels assign one thread (CUDA) or work-item (OpenCL) per element, with a bounds check for sizes that are not a multiple of the block size. CUDA uses a fixed block size of 256 threads. Two host-side variants were implemented for each API. In PyCUDA, the explicit variant allocates device memory with `mem_alloc` and moves data with `memcpy_htod` and `memcpy_dtoh`, while the `gpuarray` variant uses `gpuarray.to_gpu` and `.get`. In PyOpenCL, the buffer variant creates `cl.Buffer` objects and copies results back with `enqueue_copy`, while the array variant uses `cl.array.to_device` and `.get`.

All variants within an API launch the same kernel, so differences in end-to-end time reflect only how memory is allocated and transferred.

B. Timing

Kernel-only time covers the kernel launch alone. End-to-end time covers device allocation, the host-to-device copy,

the kernel, and the device-to-host copy. PyCUDA uses CUDA events for both measurements. PyOpenCL uses event profiling for kernel time and `time.time()` for end-to-end time. Each configuration was run 10 times and averaged. The CPU baseline uses `numpy.maximum`.

Correctness was verified against NumPy for edge cases including a single element, all-negative and all-positive inputs, a length not divisible by the block size ($N = 1,000,003$), and a 25M-element vector. All implementations passed every test.

C. Experimental Platform

Experiments were run on a Google Cloud VM. The hardware is summarized in Table II and the software stack in Table I.

TABLE I: Software stack

Component	Version / details
CUDA Toolkit	12.6.1 (nvcc, nsys, ncu)
NVIDIA driver	560.35.03
Compiler	GCC 12
Python	Python 3 (venv)
GPU libraries	PyCUDA, PyOpenCL (NVIDIA CUDA platform)
Python packages	NumPy, Matplotlib

TABLE II: Experimental platform

Item	Value
GPU	1 × NVIDIA Tesla P4 (Pascal, 8 GB GDDR5)
Machine type	GCP n1-standard-4 (4 vCPUs, 15 GB RAM)
OS	Ubuntu 22.04 LTS
Disk	50 GB persistent
Zone	us-east4-b

TABLE III: PyCUDA execution times in seconds

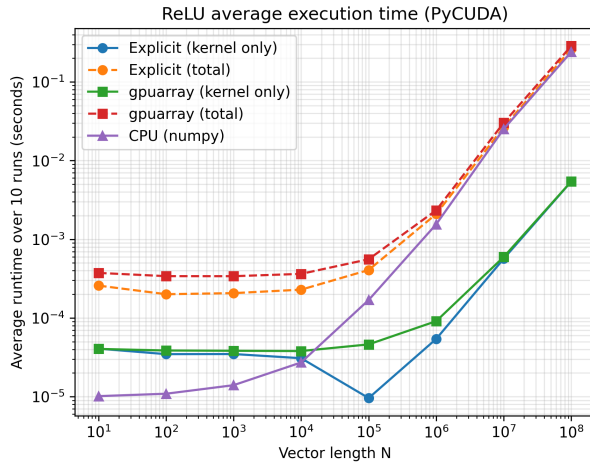
N	Explicit		gpuarray		CPU
	Kernel	Total	Kernel	Total	
10	4.076e-05	2.580e-04	4.045e-05	3.739e-04	1.016e-05
10^2	3.472e-05	2.005e-04	3.860e-05	3.402e-04	1.090e-05
10^3	3.482e-05	2.069e-04	3.830e-05	3.397e-04	1.400e-05
10^4	3.082e-05	2.291e-04	3.800e-05	3.636e-04	2.723e-05
10^5	9.600e-06	4.072e-04	4.618e-05	5.578e-04	1.692e-04
10^6	5.416e-05	2.095e-03	9.112e-05	2.340e-03	1.552e-03
10^7	5.670e-04	2.661e-02	5.969e-04	3.024e-02	2.522e-02
10^8	5.424e-03	2.626e-01	5.435e-03	2.864e-01	2.407e-01

TABLE IV: PyOpenCL execution times in seconds

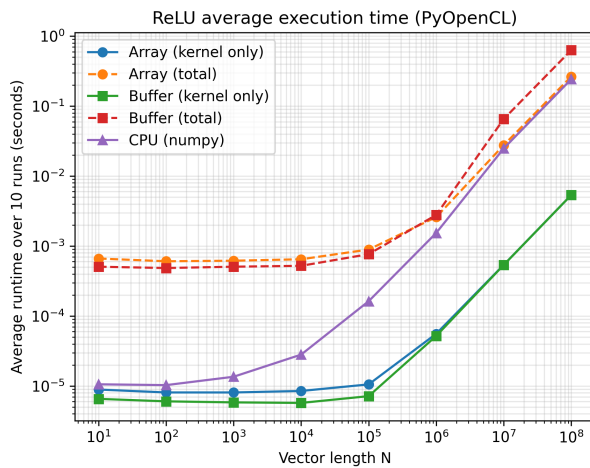
N	Array		Buffer		CPU
	Kernel	Total	Kernel	Total	Total
10^1	8.909e-06	6.639e-04	6.554e-06	5.068e-04	1.059e-05
10^2	8.093e-06	6.062e-04	6.038e-06	4.854e-04	1.030e-05
10^3	8.090e-06	6.174e-04	5.837e-06	5.069e-04	1.357e-05
10^4	8.499e-06	6.478e-04	5.734e-06	5.222e-04	2.811e-05
10^5	1.055e-05	8.920e-04	7.168e-06	7.658e-04	1.628e-04
10^6	5.581e-05	2.593e-03	5.192e-05	2.784e-03	1.538e-03
10^7	5.392e-04	2.770e-02	5.370e-04	6.546e-02	2.485e-02
10^8	5.366e-03	2.647e-01	5.367e-03	6.309e-01	2.424e-01

III. RESULTS

Fig. 1 and Tables III and IV report average execution times. All curves are roughly flat for small N , where fixed costs dominate, and grow approximately linearly once N exceeds about 10^5 .



(a) PyCUDA



(b) PyOpenCL

Fig. 1: Average ReLU execution time for the PyCUDA and PyOpenCL implementations.

A. Kernel-Only Performance

The PyCUDA kernel overtakes NumPy at about $N = 10^5$. The PyOpenCL kernel is already faster at $N = 10$. At $N = 10^8$, both GPU kernels take about 5.4 ms against 0.24 s for NumPy, a speedup of roughly $45\times$. Within each API, both memory-management variants yield nearly identical kernel times, as expected, since they launch the same kernel.

B. End-to-End Performance

NumPy is faster than every GPU variant end-to-end at all tested sizes, so no end-to-end crossover occurs. The gap narrows as N grows: at $N = 10^8$, the fastest GPU variant (PyCUDA explicit, 0.263 s) is within about 9% of NumPy (0.241 s).

C. Transfer Overhead

Table V gives the ratio of end-to-end to kernel-only time. The ratio grows with N because transfer volume scales with the input while the kernel itself is cheap.

TABLE V: Ratio of end-to-end to kernel-only time

Implementation	$N = 10$	$N = 10^8$
PyCUDA (explicit)	6.33 $\times$	48.41 $\times$
PyOpenCL (buffer)	77.33 $\times$	117.55 $\times$

D. High-Level Arrays versus Explicit Memory Management

In PyCUDA, the explicit variant is consistently faster end-to-end than gpuarray. It is about 0.024 s faster at $N = 10^8$, and the gap is smallest around $N = 10^6$. In PyOpenCL, the two variants are similar for small N but diverge beyond $N = 10^6$. At $N = 10^8$, the array interface is about 0.366 s faster than the buffer interface.

IV. DISCUSSION

A. Why the GPU Loses End-to-End

ReLU does very little work per element, so its runtime depends mostly on moving data rather than computing. The GPU computes ReLU quickly, but the input must first be copied to the GPU and the result copied back. For small N , fixed costs such as memory allocation and kernel launch dominate. For large N , the data transfers themselves dominate. In both cases, the kernel is only a small part of the total time. Running ReLU on the GPU by itself is therefore not worthwhile. It becomes worthwhile when the data is already on the GPU, for example as one layer in a neural network.

B. Interface Trade-offs

The array interfaces wrap allocation and transfer: `to_gpu()` and `to_device()` allocate device memory and copy the input, while `.get()` allocates a new host array and copies the result back. This convenience costs some control. The explicit interfaces let the programmer pre-allocate host buffers and manage transfers directly. Since kernel times match within each API, the observed end-to-end differences come from these allocation and transfer paths.

C. Synchronization

Kernel launches in both APIs are asynchronous: control returns to the host while the device executes. Correctness of the output does not require an explicit synchronization, because the device-to-host copy is blocking and is enqueued after the kernel on the same stream or queue. Accurate timing does require one. `.synchronize()` in PyCUDA and `.wait()` in PyOpenCL ensure that an event has completed before its timestamp is read. Without them, the measured interval could end before the GPU work finishes.

V. LIMITATIONS AND FUTURE WORK

Index range. The CUDA kernel uses a signed `int` index, which overflows beyond $2^{31} - 1$ elements. A grid-stride loop, advancing each thread by `blockDim.x * gridDim.x`, would let a fixed-size grid cover arbitrarily large inputs.

Device memory capacity. Inputs larger than device memory cannot be allocated at once. Processing the input in chunks, reusing a single device buffer for each transfer–compute–copy cycle, removes this limit. Overlapping chunks with multiple streams could additionally hide part of the transfer cost.

Measurement. The PyCUDA explicit kernel time at $N = 10^5$ is lower than at smaller sizes, which likely reflects measurement noise. Warm-up runs and reporting variance alongside the mean would make the results more robust.

VI. CONCLUSION

On a Tesla P4, a GPU ReLU kernel is up to about $45\times$ faster than NumPy in pure computation, but host–device transfers make it slower end-to-end at every tested size. For memory-bound operations like ReLU, the benefit of GPU offload depends far more on keeping data resident on the device than on kernel speed.

ACKNOWLEDGMENT

This work was completed as a homework assignment for EECS E4750: Heterogeneous Computing for Signal and Data Processing at Columbia University, taught by Prof. Zoran Kostić. The assignment template, test harness, benchmarking code and the OpenCL kernel were provided by the course. The CUDA kernel and all host code were developed by the author. This report was written with the assistance of AI; all experimental results were produced and verified by the author.